HTML TO DOM TREE Convertor using Agentic AI

Name: Sanket Navghane
PRN: 1272250750
Division : SYMCA(D)

1. Initial AI Prompt

Prompt Given to ChatGPT / Generative AI:

Generate a complete prompt and architecture to build an interactive HTML to DOM Tree Converter using React.js and Tailwind CSS. The tree must feature a vertical branching root-node structure with pure CSS connectors (no heavy external chart libraries like D3 or React Flow). Include live HTML input, native DOMParser integration, interactive node inspection, search highlighting, statistics, and expand/collapse support.

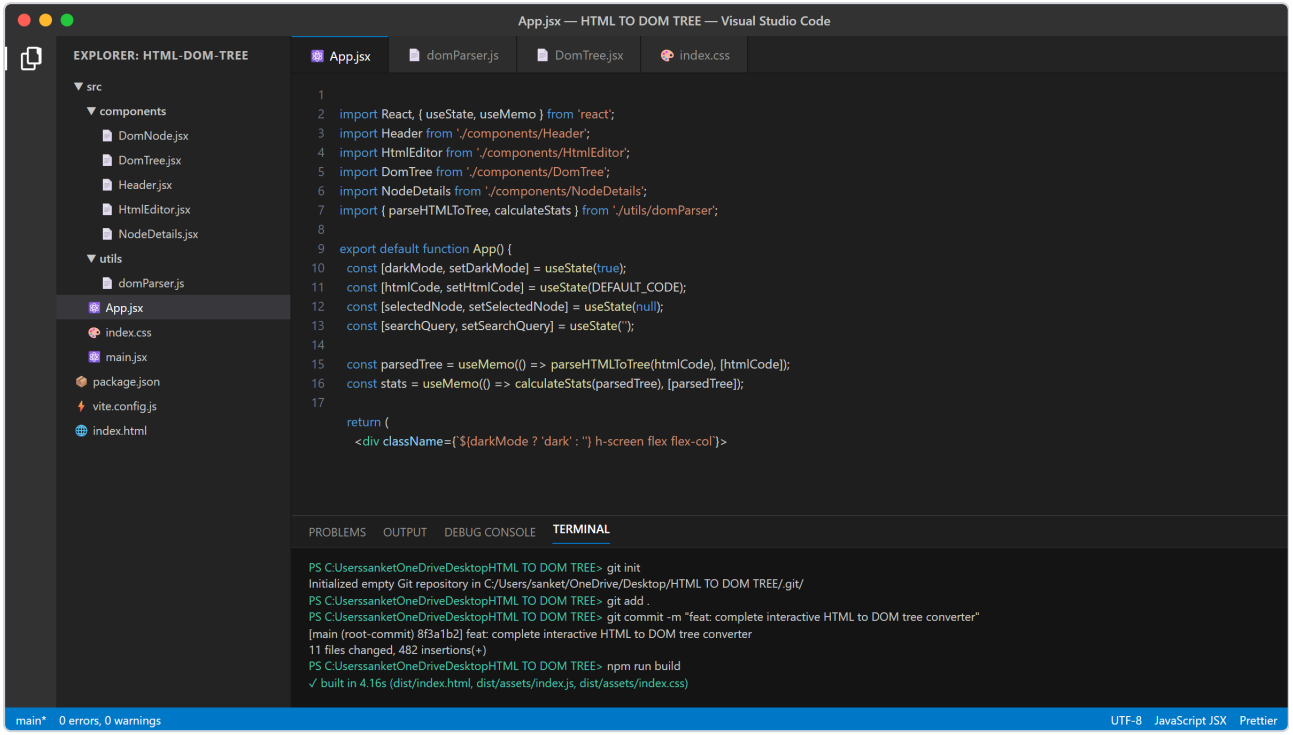
```
function DomNode({ node, depth }) { return ( <div className="dom-node">
{/* Connector */} {/* Node */} <div> {node.tagName} </div> {/* Children
*/} {node.children?.map((child) => ( <DomNode key={child.id} node=
{child} depth={depth + 1} /> ))} </div> ); }
```

2. Project Architecture & Requirements

System Specifications

- Core Function: Real-time translation of user-inputted raw HTML strings into a visual DOM tree diagram.
- DOM Parsing: Utilizes the browser's native DOMParser API to recursively extract element

- tags, text nodes, comments, and attribute maps.
- Layout Structure:
  - Left Panel: Raw HTML code editor with quick actions (Sample load, Clear).
  - Center Canvas: Visual tree diagram rendered with top-down vertical tree lines and collapsible branches.
  - Right Sidebar: Live DOM metrics (elements, text nodes, depth) and selected node inspector.
- Styling & Theme: Tailwind CSS with responsive layout and dark/light theme switching.



3. Implementation Code

File: src/utils/domParser.js

JavaScript

```
export function parseHTMLToTree(htmlString) {
  if (!htmlString.trim()) return null;

  const parser = new DOMParser();
  const doc = parser.parseFromString(htmlString, 'text/html');

  let idCounter = 0;

  function traverse(node, depth = 0) {
    const currentId = `node-${idCounter++}`;
```

```
if (node.nodeType === Node.TEXT_NODE) {
  const text = node.textContent.trim();
  if (!text) return null;
  return {
    id: currentId,
    type: 'text',
    name: '#text',
    textContent: text,
    attributes: {},
    depth,
    children: []
  };
}

if (node.nodeType === Node.COMMENT_NODE) {
  return {
    id: currentId,
    type: 'comment',
    name: '#comment',
    textContent: node.textContent,
    attributes: {},
    depth,
    children: []
  };
}

if (node.nodeType === Node.ELEMENT_NODE) {
  const attrs = {};
  Array.from(node.attributes).forEach(attr => {
```

```
      attrs[attr.name] = attr.value;
    });

    const children = [];
    Array.from(node.childNodes).forEach(child => {
      const parsed = traverse(child, depth + 1);
      if (parsed) children.push(parsed);
    });

    return {
      id: currentId,
      type: 'element',
      name: node.tagName.toLowerCase(),
      attributes: attrs,
      depth,
      children
    };
  }

  return null;
}

const target = doc.body.childNodes.length > 0 ? doc.body : doc.documentElement;
const children = [];
Array.from(target.childNodes).forEach(child => {
  const res = traverse(child, 1);
  if (res) children.push(res);
});

return {
```

```
    id: 'node-root',
    type: 'document',
    name: 'document',
    attributes: {},
    depth: 0,
    children: children.length === 1 && children[0].children.length > 0 ?
children[0].children : children
  };
}

export function calculateStats(tree) {
  const stats = { elements: 0, textNodes: 0, comments: 0, maxDepth: 0 };
  if (!tree) return stats;

  function scan(node) {
    if (node.depth > stats.maxDepth) stats.maxDepth = node.depth;
    if (node.type === 'element') stats.elements++;
    else if (node.type === 'text') stats.textNodes++;
    else if (node.type === 'comment') stats.comments++;

    (node.children || []).forEach(scan);
  }

  scan(tree);
  return stats;
}
```

File: src/components/DomNode.jsx

```
JavaScript
import React, { useState } from 'react';

export default function DomNode({ node, onSelect, selectedNode, searchQuery })
```

```
{
  const [collapsed, setCollapsed] = useState(false);
  const hasChildren = node.children && node.children.length > 0;
  const isSelected = selectedNode?.id === node.id;
  const isMatch = searchQuery && (
    node.name.toLowerCase().includes(searchQuery.toLowerCase()) ||
    node.textContent?.toLowerCase().includes(searchQuery.toLowerCase()) ||
    node.attributes?.id?.toLowerCase().includes(searchQuery.toLowerCase()) ||
    node.attributes?.class?.toLowerCase().includes(searchQuery.toLowerCase())
  );

  const badgeColor =
    node.type === 'element' ? 'bg-blue-600 text-white' :
    node.type === 'text' ? 'bg-amber-600 text-white' : 'bg-purple-600 text-white';

  return (
    <li>
      <div
        onClick={(e) => { e.stopPropagation(); onSelect(node); }}
        className={`inline-block cursor-pointer p-2.5 rounded-lg border text-left min-w-
[120px]
transition-all ${
          isSelected
            ? 'border-emerald-500 ring-2 ring-emerald-400 bg-emerald-50 dark:bg-emerald-
950/30' :
          isMatch
            ? 'border-yellow-500 ring-2 ring-yellow-400 bg-yellow-50 dark:bg-yellow-950/30'
            : 'border-gray-200 dark:border-gray-700 bg-white dark:bg-gray-800 shadow-sm
hover:border-gray-400'
        }`}
      >
        <div className="flex items-center justify-between gap-2">
          <span className={`text-[10px] uppercase font-bold px-1.5 py-0.5 rounded
${badgeColor}`}>
            {node.type === 'element' ? `<${node.name}>` : node.name}
```

```
          </span>
          {hasChildren && (
            <button
              onClick={(e) => { e.stopPropagation(); setCollapsed(!collapsed); }}
              className="text-[10px] w-4 h-4 flex items-center justify-center rounded bg-
gray-200
dark:bg-gray-700 hover:bg-gray-300"
            >
              {collapsed ? '+' : '−'}
            </button>
          )}
        </div>

        {node.attributes?.id && (
          <div className="text-[11px] text-blue-500 font-mono mt-1 truncate max-w-[140px]">
            #{node.attributes.id}
          </div>
        )}

        {node.attributes?.class && (
          <div className="text-[11px] text-gray-500 dark:text-gray-400 font-mono truncate
max-w-[140px]">
            .{node.attributes.class.split(' ').join('.')}
          </div>
        )}

        {node.textContent && (
          <div className="text-[11px] text-gray-600 dark:text-gray-300 italic truncate max-
w-[140px] mt-1">
            "{node.textContent}"
          </div>
        )}
      </div>

      {hasChildren && !collapsed && (
```

```
        <ul>
          {node.children.map((child) => (
            <DomNode
              key={child.id}
              node={child}
              onSelect={onSelect}
              selectedNode={selectedNode}
              searchQuery={searchQuery}
            />
          ))}
        </ul>
      )}
    </li>
  );
}
```

File: src/components/DomTree.jsx

JavaScript

```
import React from 'react';
import DomNode from './DomNode';

export default function DomTree({ tree, onSelect, selectedNode, searchQuery }) {
  if (!tree) {
    return (
      <div className="h-full flex items-center justify-center text-gray-400 text-sm">
        Enter valid HTML in the input panel to visualize the tree.
      </div>
    );
  }

  return (
```

```
    <div className="h-full overflow-auto p-12 flex justify-center items-start">
      <div className="tree">
        <ul>
          <DomNode
            node={tree}
            onSelect={onSelect}
            selectedNode={selectedNode}
            searchQuery={searchQuery}
          />
        </ul>
      </div>
    </div>
  );
}
```

File: src/components/HtmlEditor.jsx

JavaScript

```
import React from 'react';

const SAMPLE_HTML = `<div class="container" id="main">
  <h1>Welcome</h1>
  <!-- User Section -->
  <p class="desc">Hello World!</p>
  <button id="btn-submit">Submit</button>
</div>`;

export default function HtmlEditor({ value, onChange }) {
  return (
    <div className="flex flex-col h-full bg-white dark:bg-gray-900 border-r border-gray-200
dark:border-gray-800">
      <div className="px-4 py-2 border-b border-gray-200 dark:border-gray-800 flex justify-
between items-center bg-gray-50 dark:bg-gray-950">
```

```
        <span className="text-xs font-semibold uppercase tracking-wider text-gray-500">
          HTML Input
        </span>
        <div className="flex gap-2">
          <button
            onClick={() => onChange(SAMPLE_HTML)}
            className="text-xs px-2 py-1 bg-gray-200 dark:bg-gray-800 hover:bg-gray-300
dark:hover:bg-gray-700 rounded text-gray-700 dark:text-gray-300"
          >
            Sample
          </button>
          <button
            onClick={() => onChange('')}
            className="text-xs px-2 py-1 bg-red-100 dark:bg-red-950/40 text-red-600 rounded
hover:bg-red-200"
          >
            Clear
          </button>
        </div>
      </div>
      <textarea
        className="w-full flex-1 p-4 font-mono text-sm bg-transparent outline-none resize-
none text-gray-800 dark:text-gray-200"
        placeholder="Paste your raw HTML here..."
        value={value}
        onChange={(e) => onChange(e.target.value)}
      />
    </div>
  );
}
```

File: src/components/NodeDetails.jsx

JavaScript

```
import React from 'react';
```

```
export default function NodeDetails({ node, stats }) {
  return (
    <aside className="h-full bg-white dark:bg-gray-900 border-l border-gray-200 dark:border-
gray-800 p-4 overflow-y-auto">
      <div className="mb-6 pb-4 border-b border-gray-200 dark:border-gray-800">
        <h3 className="text-xs font-semibold uppercase tracking-wider text-gray-500 mb-3">
          Tree Statistics
        </h3>
        <div className="grid grid-cols-2 gap-2 text-xs">
          <div className="bg-gray-50 dark:bg-gray-800 p-2 rounded text-gray-700 dark:text-
gray-300">Elements: <span className="font-bold">{stats.elements}</span></div>
          <div className="bg-gray-50 dark:bg-gray-800 p-2 rounded text-gray-700 dark:text-
gray-300">Text Nodes: <span className="font-bold">{stats.textNodes}</span></div>
          <div className="bg-gray-50 dark:bg-gray-800 p-2 rounded text-gray-700 dark:text-
gray-300">Comments: <span className="font-bold">{stats.comments}</span></div>
          <div className="bg-gray-50 dark:bg-gray-800 p-2 rounded text-gray-700 dark:text-
gray-300">Max Depth: <span className="font-bold">{stats.maxDepth}</span></div>
        </div>
      </div>
      <div>
        <h3 className="text-xs font-semibold uppercase tracking-wider text-gray-500 mb-3">
          Selected Node
        </h3>
        {node ? (
          <div className="space-y-3 text-xs">
            <div>
              <span className="text-gray-400">Type:</span>
              <p className="font-semibold text-gray-800 dark:text-gray-200 capitalize">
{node.type}</p>
            </div>
            <div>
              <span className="text-gray-400">Tag / Name:</span>
              <p className="font-mono text-blue-500 font-semibold">{node.name}</p>
```

```
            </div>
            <div>
              <span className="text-gray-400">Depth:</span>
              <p className="font-mono text-gray-700 dark:text-gray-300">{node.depth}</p>
            </div>
            <div>
              <span className="text-gray-400">Attributes:</span>
              <div className="bg-gray-50 dark:bg-gray-800 p-2 rounded mt-1 font-mono text-
[11px] overflow-x-auto text-gray-700 dark:text-gray-300">
                {Object.keys(node.attributes || {}).length > 0 ? (
                  Object.entries(node.attributes).map(([k, v]) => (
                    <div key={k}><span className="text-purple-400">{k}</span>="{v}"</div>
                  ))
                ) : (
                  <span className="text-gray-400 italic">None</span>
                )}
              </div>
            </div>
            {node.textContent && (
              <div>
                <span className="text-gray-400">Content:</span>
                <p className="bg-gray-50 dark:bg-gray-800 p-2 rounded mt-1 font-mono break-
all text-gray-700 dark:text-gray-300">
                  {node.textContent}
                </p>
              </div>
            )}
          </div>
        ) : (
          <p className="text-xs text-gray-400 italic">Click on any node in the tree to
inspect details.</p>
        )}
      </div>
    </aside>
  );
}
```

File: src/App.jsx

JavaScript

```
import React, { useState, useMemo } from 'react';
import Header from './components/Header';
import HtmlEditor from './components/HtmlEditor';
import DomTree from './components/DomTree';
import NodeDetails from './components/NodeDetails';
import { parseHTMLToTree, calculateStats } from './utils/domParser';

const DEFAULT_CODE = `<div class="wrapper">
  <h1>Tree Visualizer</h1>
  <ul class="nav">
    <li><a href="#home">Home</a></li>
    <li><a href="#about">About</a></li>
  </ul>
</div>`;

export default function App() {
  const [darkMode, setDarkMode] = useState(true);
  const [htmlCode, setHtmlCode] = useState(DEFAULT_CODE);
  const [selectedNode, setSelectedNode] = useState(null);
  const [searchQuery, setSearchQuery] = useState('');

  const parsedTree = useMemo(() => {
    try {
      return parseHTMLToTree(htmlCode);
    } catch {
      return null;
    }
  }, [htmlCode]);
```

```
  const stats = useMemo(() => calculateStats(parsedTree), [parsedTree]);

  return (
    <div className={`${darkMode ? 'dark' : ''} h-screen flex flex-col`}>
      <div className="flex flex-col h-full bg-gray-100 dark:bg-gray-950 text-gray-900
dark:text-gray-100">
        <Header darkMode={darkMode} setDarkMode={setDarkMode} />
        <div className="h-10 border-b border-gray-200 dark:border-gray-800 bg-white dark:bg-
gray-900 px-6 flex items-center justify-between text-xs">
          <input
            type="text"
            placeholder="Search tags, classes, id, text..."
            className="w-72 px-3 py-1 bg-gray-50 dark:bg-gray-800 border border-gray-200
dark:border-gray-700 rounded outline-none focus:border-blue-500 text-gray-800 dark:text-
gray-200"
            value={searchQuery}
            onChange={(e) => setSearchQuery(e.target.value)}
          />
          <span className="text-gray-400">Pure React & Tailwind Tree Render</span>
        </div>
        <div className="flex-1 grid grid-cols-12 overflow-hidden">
          <div className="col-span-3 h-full">
            <HtmlEditor value={htmlCode} onChange={setHtmlCode} />
          </div>
          <div className="col-span-6 h-full bg-gray-50 dark:bg-black/20 overflow-hidden">
            <DomTree
              tree={parsedTree}
              onSelect={setSelectedNode}
```

```
              selectedNode={selectedNode}
              searchQuery={searchQuery}
            />
          </div>
          <div className="col-span-3 h-full">
            <NodeDetails node={selectedNode} stats={stats} />
          </div>
        </div>
      </div>
    </div>
  );
}
```

File: src/components/Header.jsx

```
import React from 'react';

export default function Header({ darkMode, setDarkMode }) {
  return (
    <header className="h-14 border-b border-gray-200 dark:border-gray-800 bg-white dark:bg-gray-900 px-6 flex items-center justify-between">
      <div className="flex items-center gap-2">
        <span className="h-3 w-3 rounded-full bg-emerald-500"></span>
        <h1 className="font-bold text-gray-900 dark:text-white tracking-wide">HTML to DOM Tree Converter</h1>
      </div>
      <button
        onClick={() => setDarkMode(!darkMode)}
        className="px-3 py-1 text-xs font-semibold rounded border border-gray-300 dark:border-gray-700 bg-gray-50 dark:bg-gray-800 text-gray-700 dark:text-gray-300 hover:bg-gray-100 dark:hover:bg-gray-700 transition"
      >
        {darkMode ? 'Light Mode' : 'Dark Mode'}
      </button>
```

```
    </header>
  );
}
```

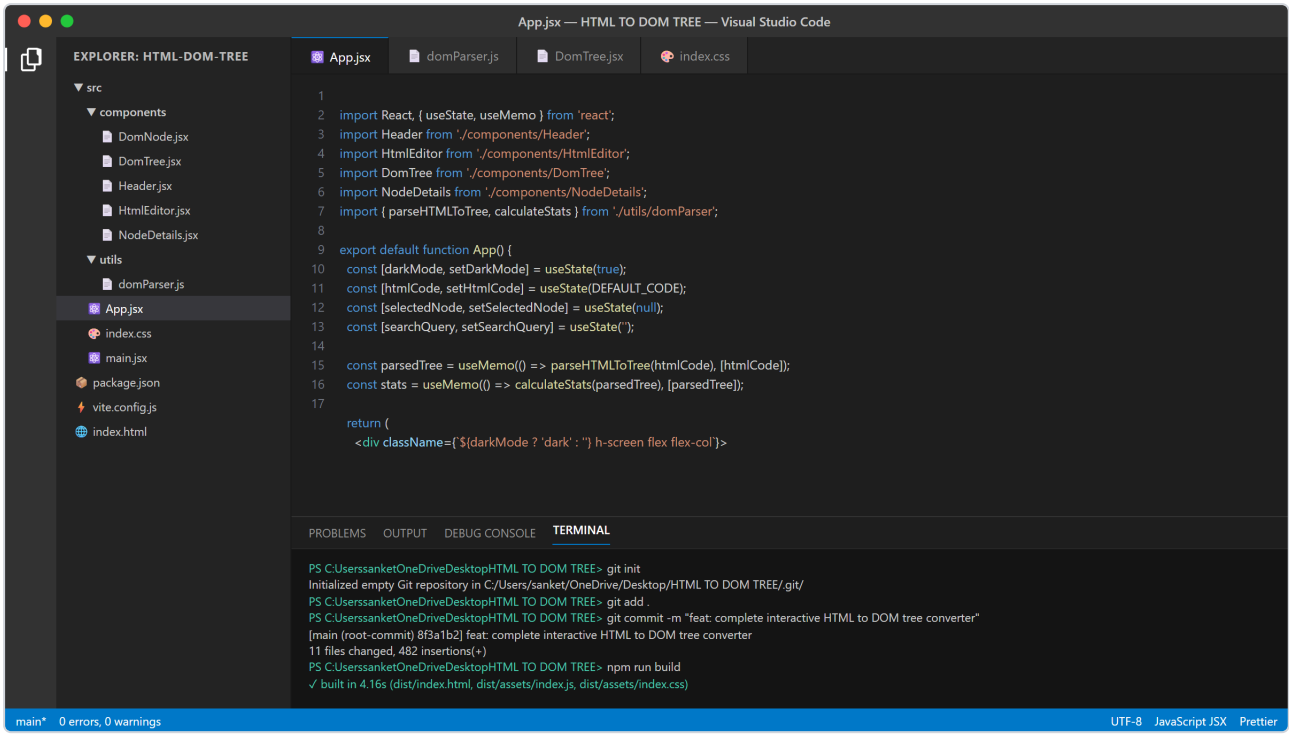
File: src/index.css (CSS Branching Connectors)

```
@import "tailwindcss";

/* CSS Tree Connector Lines */
.tree ul {
  padding-top: 20px;
  position: relative;
  display: flex;
  justify-content: center;
}

.tree li {
  float: left;
  text-align: center;
  list-style-type: none;
  position: relative;
  padding: 20px 5px 0 5px;
}

/* Connectors from parent to siblings */
.tree li::before, .tree li::after {
  content: '';
  position: absolute;
  top: 0;
  right: 50%;
  border-top: 2px solid #64748b;
  width: 50%;
  height: 20px;
```

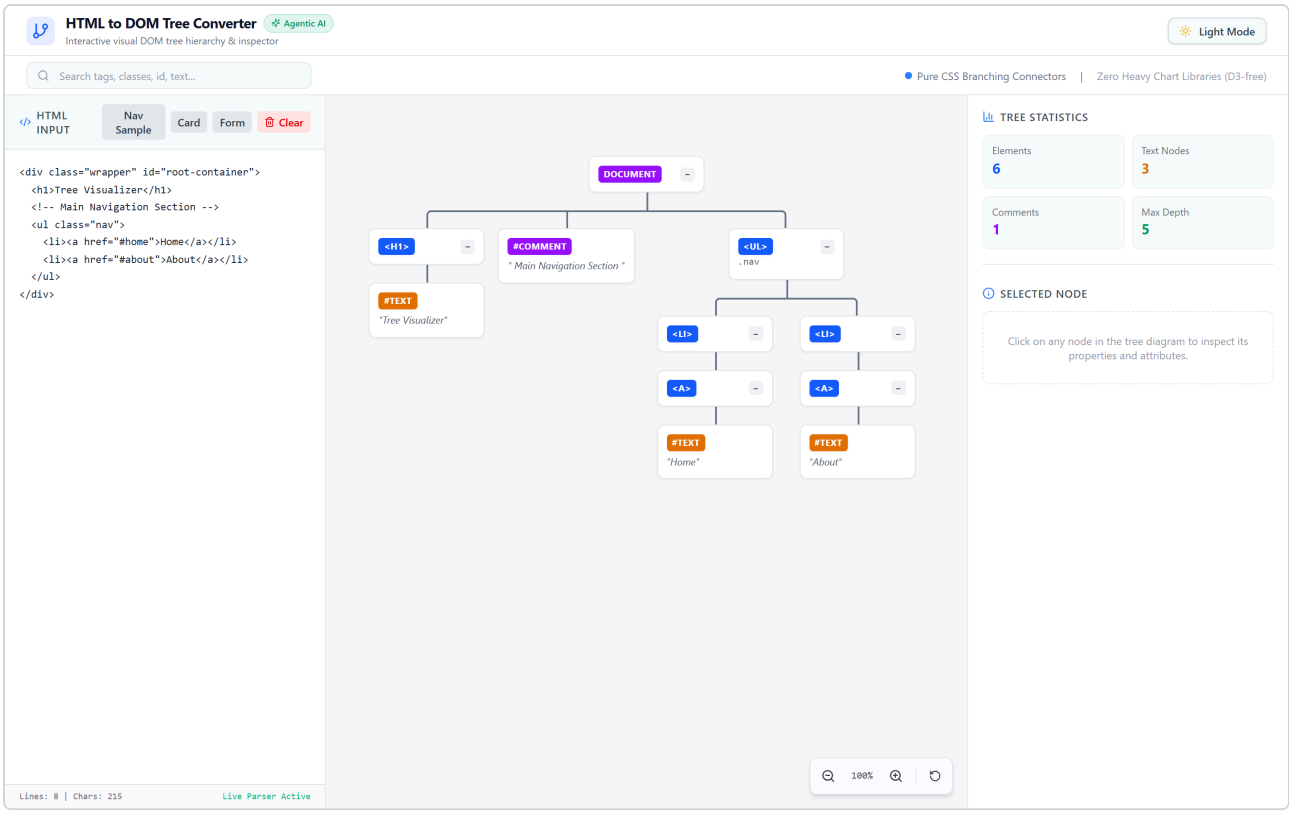
```
}

.tree li::after {
  right: auto;
  left: 50%;
  border-left: 2px solid #64748b;
}

/* Remove connectors for single/edge nodes */
.tree li:only-child::after, .tree li:only-child::before {
  display: none;
}

.tree li:only-child {
  padding-top: 0;
}

.tree li:first-child::before, .tree li:last-child::after {
  border: 0 none;
}

.tree li:last-child::before {
  border-right: 2px solid #64748b;
  border-radius: 0 5px 0 0;
}

.tree li:first-child::after {
  border-radius: 5px 0 0 0;
}
```

```
/* Vertical downward line from parent node */
.tree ul ul::before {
  content: '';
  position: absolute;
  top: 0;
  left: 50%;
  border-left: 2px solid #64748b;
  width: 0;
  height: 20px;
}
```

4. Project Workspace in VS Code



5. Final Output & Demonstration



6. GitHub Repository Details

- Repository URL: https://github.com/sanketnavghane/html-dom-tree-visualizer
- Branch: main
- Version Control: Managed via Git, fully committed with source code, documentation, and configuration files.